Test IEEE Conference Template

Thomas Kühn, Edward Sabinus
Software Engineering and Programming Languages Group
Martin-Luther University Halle-Wittenberg
Halle, Germany
0000-0001-7312-2891

Uwe Aßmann
Softwaretechnik
Technische Universität Dresden
Dresden, Germany
uwe.assmann@tu-dresden.de

***Abstract*—Role-based modeling has been investigated for over 35 years as a promising paradigm to model complex, dynamic systems. Although current software systems are characterized by increasing complexity and context-dependence, all this research had almost no influence on current software development practice, still being discussed in recent literature.**

***Index Terms*—Role-based Modeling, Role-based Programming**

I. Introduction

Charles W. Bachmann was the first researcher to investigate roles back in the year 1977. He proposed role-based modeling [1] to capture both context-dependent and collaborative behavior of objects. Since then, a large variety of approaches in different research areas, ranging from data modeling [1]–[3] via conceptual modeling [4], [5] through to programming languages [6]–[9] emerged. More importantly, because current software systems are characterized by increased complexity and context-dependence [10], there is a strong demand for new concepts beyond object-oriented design. Although mainstream object-oriented modeling languages are good at capturing a system's structure, they lack ways to model the systems behavior, as it dynamically emerges through collaborating objects [11]. In turn, roles are a natural concept capturing the behavior of participants in a collaboration. Moreover, roles permit the specification of interactions independent from the interacting objects. Similarly, more recent approaches use roles to capture context-dependent properties of objects [3], [7]. The notion of roles can help to tame the increased complexity and context-dependence. Despite all that, these years of research had almost no influence on current software development practice.

☐ This is a test
☒ This is another test

To enable the practical application of roles, two major requirements have to be fulfilled. First, besides the intuitive semantics underlying the role concept, its notions must be formally specified to create a coherent understanding of relational and context-dependent roles. Second, to permit scalability of role-based modeling languages automatic mechanisms to validate the well-formedness and consistency are required. Therefore, a formal model is indispensable. In sum, a major blocking factor for the practical application of roles is their lack of a consistent formal model. Thus, this paper provides a formal model for a role-based modeling language supporting both natures of roles. This allows for the formal and automatic verification of well-formedness, compliance, and validity of models at design time and their instances at runtime.

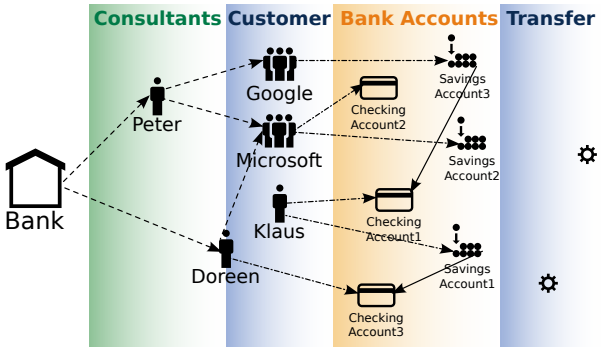


Figure 1. Overview of the banking application

The paper is structured as follows: section II introduces a small running example used for the remainder of the paper. Afterwards, section III elaborates on the nature of roles and surveys contemporary role-based modeling languages. The main contribution of this paper, ...

II. Running Example

Before diving into role-based modeling, we first employ the real world scenario of a small banking application, extracted from [11]. A bank is a financial institution that employs consultants and provides banking services to their customers, who are either persons or companies. Customers can be advised by consultants, own several savings and checking accounts, and perform transactions. Transactions encapsulate the process of transferring money from exactly one source account to one target account. They are initiated by a customer, however, managed and executed by the bank. Additionally, financial regulations require that consultants should not advise themselves as a customer and checking accounts must have exactly one owner whereas savings accounts can have multiple owners. Furthermore, no account can be both a checking and a savings account as well as both the source and target of the same transaction. Figure 1 illustrates this scenario.

This is a test text referencing Reenskaug and Coplien [11] and Listing 1.

Listing 1. Test caption

```
record P(int lvl);
int one()           { return 1; }
int up(int x)       { return x+1; }
P player()          { return new P(0); }
P attack(P p)       { return new P(p.lvl()+1); }
P spell(P p)        { return new P(p.lvl()+2); }
P fusion(P p, P q) { return new P(p.lvl()+q.lvl()); }
int level(P p)      { return p.lvl(); }
```

Table I
CLASSIFYING FEATURES OF ROLES, EXTRACTED FROM [4], [12].

Nr.	**Feature of Roles** ***(Meta level)***
1.	Roles have properties and behaviors *(M1, M0)*
2.	Roles depend on relationships *(M1)*
3.	Objects may play different roles simultaneously *(M1, M0)*
4.	Objects may play the same role (type) several times *(M0)*
5.	Objects may acquire and abandon roles dynamically *(M0)*
6.	The sequence of role acquisition and removal may be restricted *(M1, M0)*
7.	Unrelated objects can play the same role *(M1)*
8.	Roles can play roles *(M1, M0)*
9.	Roles can be transferred between objects *(M0)*
10.	The state of an object can be role-specific *(M0)*
11.	Features of an object can be role-specific *(M1)*
12.	Roles restrict access *(M0)*
13.	Different roles may share structure and behavior *(M1)*
14.	An object and its roles share identity *(M0)*
15.	An object and its roles have different identities *(M0)*
16.	Relationships between roles can be constrained *(M1)*
17.	There may be constraints between relationships *(M1)*
18.	Roles can be grouped and constrained together *(M1)*
19.	Roles depend on compartments *(M1, M0)*
20.	Compartments have properties and behaviors *(M1, M0)*
21.	A role can be part of several compartments *(M1, M0)*
22.	Compartments may play roles like objects *(M1, M0)*
23.	Compartments may play roles which are part of themselves *(M1, M0)*
24.	Compartments can contain other compartments *(M1, M0)*
25.	Different compartments may share structure and behavior *(M1)*
26.	Compartments have their own identity *(M0)*

III. NATURE OF ROLES

Roles are not a new concept in modeling languages, however, the only generally accepted feature of roles is that they can be played by unrelated objects [4], [12]. The role of a *customer* of a bank, for instance, can be played by either a *person* or a *company*, regardless of them being related or not. Besides that, [4] and [12] have shown that there is no common understanding of roles in the literature. They identified 26 classifying features attributed to roles, shown in Table I. For now, two of these features are important, because they help to classify approaches to support either the relational nature or context-dependent nature of roles, namely Feature 2 and 19. Additionally, several features indicate that roles and role models are subject to constraints, e.g., Features 6, 16, 17, and 18. Accordingly, the following discussion is trisected.

A. Relational Nature

Modeling languages usually feature some notion of role. Consider, ER [13] and UML [14], where roles denote the named ends of relationships or associations.

Either way, they neither have properties (Feature 1) nor the ability to be played by unrelated objects (Feature 7). In the case of UML, one could argue that this can be resolved by modeling roles as classes and use inheritance to relate them to their players. This, however, fails to capture the intended dynamism of roles and results in exponentially many classes [4]. Several role-based modeling languages [1], [2], [4], [15], [16] introduced roles tied to relationships as first-class citizens. Hence, these approaches can represent the *advises* relationship between *consultants* and *customers* and the latter be played by either *persons* or *companies*. These languages assume that all roles and relationships are equally relevant to an object's properties. Thus, there is no notion of context, scope, or institution on which roles and relationships depend on. In most approaches (except [2], [15]) relationships cannot play roles themselves. In the banking application, the transaction must be modeled as a relationship, but cannot be tied to the specific bank owning this transaction. This, in turn, prevents reusing the notion of a financial transaction, as it is now tied to the modeled bank. To resolve this dilemma it must be understood that relationships themselves are context-dependent.

B. Context-Dependent Nature

To incorporate the missing contextual-dependency of roles, recent role-based modeling languages [11], [17], [18] focused on the context-dependent nature. These approaches introduce some sort of *context* to encapsulate the roles relevant to a certain situation or interaction. Due to the fact that the term itself is massively overloaded, several researchers introduced other terms to denote the context of a role, e.g., *environment* [19], *institution* [20], *ensemble* [18], and *compartment* [12]. Henceforth, the term *compartment* is used because its definition, as *"objectified collaboration with a limited number of participating roles and a fixed scope"* [12], encompasses all of the other notions. In conclusion, these approaches capture the contextual dependence of roles by making the compartments their definitional boundary. For the banking example, the *transaction* can be considered a compartment, as it captures the transfer of money from a *source* to a *target* account. While most of these approaches assume that roles depend on (some kind of) compartment (Feature 19) and have properties (Feature 20), only few consider compartments as objects able to play roles themselves (Feature 22). The latter is crucial to ensure that the *transaction* compartment (instance) can be owned by a *bank* compartment (instance) managing its creation and execution. Nevertheless, most approaches relied solely on compartments [11], [17], [19], [20] and did not include context-dependent relationships between roles.

C. Constraining Role Models

So far the discussion revolved around the nature of roles in role-based modeling languages and did not cover their capabilities to specify particular constraints for roles and relationships. Typically, modeling languages support various

constraints on relationships (Feature 16). *Cardinality constraints*, for instance, are featured in most modeling languages, e.g., ER [13] and UML [14]. They limit the number of entities related by a relationship. In our example, the *owns checking account* relationship must be constrained with cardinality *one* on the *customer* side and *zero-to-many* on the *checking account* to ensure that checking accounts are owned by exactly one customer. *Intra-relationship constraints* [15] represent mathematical constraints for relations that additionally constrain relationships. The *advises* relationship, for instance, can be constrained to be *irreflexive* ensuring that persons playing the *consultant* role cannot advise themselves as a *customer*. *Inter-relationship constraints* [2] are constraints between individual relationships (Feature 17). They can be used to define subsets or disjunctions between relationships. An example for the former is a *father son* relationship required to be a subset of the *parent child* relationship. While these constraints limit relationships, there are two kinds of constraints for roles. *Role constraints* limit the types of roles that can be played simultaneously by one object. They include notions to prohibit or require another role to be played if the other role is played [21]. In our example, the statement that an *account* cannot be a *checking account* and a *savings account* at the same time can be expressed with a *role-prohibition* between these roles. Moreover, [12], [19] suggested to employ notions to group and constrain roles together. In contrast, *occurrence constraints* [18], [19] limit the number of roles present in a compartment, i.e., the number of role instances of a particular type in a compartment instance. Consider the *transaction* compartment that requires the presence of exactly one *source* and one *target* role. Although various role-based modeling languages have introduced different kinds of constraints, no approach has included all of them into one coherent model.

IV. Conclusion

This work is based on the classification of roles by [4], [12] and the family of role-based modeling languages, proposed in [12]. However, as our goal was to provide a comprehensive definition for role-based modeling, the presented model only encompasses simple, formal definitions for the type and instance level. Additionally, it comprises definitions for cardinality and intra-relationship constraints, as well as the newly introduced role groups. Moreover, we have shown that our formal model is suitable for both manual and automatic evaluation of well-formedness, compliance, and validity. Finally, the provided reference implementations can be used to apply, further explore, and extend our role-based modeling language.

In future, we will augment the formal model to include inheritance for naturals and compartments, as well as global role constraints. Moreover, our goal is to use the formal model as a reference to develop a customizable family of formal role-based modeling languages with full tool support, e.g.: graphical editor, schema, and code generators. In fact, our goal is to include these rules into an integrated development environment for role-based systems to check the well-formedness of role models as well as the compliance and validity of their instances.

Acknowledgment

Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author and do not necessarily reflect the views of the National Science Foundation.

References

[1] C. W. Bachman, M. Daya, C. W. Bachman, and M. Daya, "The role concept in data models," in *Proceedings of the third international conference on Very Large Data Bases*, vol. 3, 1977, pp. 464–476.

[2] T. Halpin, "ORM 2," in *On the Move to Meaningful Internet Systems 2005: OTM 2005 Workshops*, Springer, 2005, pp. 676–687.

[3] M. Liu and J. Hu, "Information networking model," in *Conceptual Modeling-ER 2009*, Springer, 2009, pp. 131–144.

[4] F. Steimann, "On the representation of roles in object-oriented and conceptual modelling," *Data & Knowledge Engineering*, vol. 35, no. 1, pp. 83–106, 2000.

[5] G. Guizzardi and G. Wagner, "Conceptual simulation modeling with onto-uml," in *Proceedings of the Winter Simulation Conference*, Winter Simulation Conference, 2012, p. 5.

[6] M. Baldoni, G. Boella, and L. van der Torre, "powerJava: Ontologically founded roles in object oriented programming languages," in *SAC*, H. Haddad, Ed., ACM, 2006, pp. 1414–1418.

[7] S. Herrmann, "Programming with roles in ObjectTeams/Java.," *AAAI Fall Symposium Roles, an interdisciplinary perspective*, 2005.

[8] S. Balzer, T. Gross, and P. Eugster, "A relational model of object collaborations and its use in reasoning about relationships," in *ECOOP*, E. Ernst, Ed., ser. Lecture Notes in Computer Science, vol. 4609, Springer, 2007, pp. 323–346, ISBN: 978-3-540-73588-5.

[9] M. Pradel and M. Odersky, "Scala roles: Reusable object collaborations in a library," in *Software and Data Technologies*, Springer, 2009, pp. 23–36.

[10] S. Murer, C. Worms, and F. J. Furrer, "Managed evolution," *Informatik-Spektrum*, vol. 31, no. 6, pp. 537–547, 2008.

[11] T. Reenskaug and J. O. Coplien, "The dci architecture: A new vision of object-oriented programming," *An article starting a new blog:(14pp) http://www. artima. com/articles/dci_vision. html*, 2009.

[12] T. Kühn, M. Leuthäuser, S. Götz, C. Seidl, and U. Aßmann, "A metamodel family for role-based modeling and programming languages," English, in *Software Language Engineering*, ser. Lecture Notes in Computer Science, vol. 8706, Springer, 2014, pp. 141–160.

[13] P. Chen, “The entity-relationship model - toward a unified view of data,” *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, 1976.

[14] J. Rumbaugh, R. Jacobson, and G. Booch, *The Unified Modelling Language Reference Manual*, 1st. Addison-Wesley, 1999.

[15] S. Balzer and T. R. Gross, “Verifying multi-object invariants with relationships,” in *ECOOP 2011–Object-Oriented Programming*, Springer, 2011, pp. 358–382.

[16] T. Jäkel, T. Kühn, S. Hinkel, H. Voigt, and W. Lehner, “Relationships for dynamic data types in RSQL,” in *Datenbanksysteme für Business, Technologie und Web (BTW)*, 2015.

[17] V. Genovese, “A meta-model for roles: Introducing sessions,” in *Proceedings of the 2nd Workshop on Roles and Relationships in Object Oriented Programming, Multiagent Systems, and Ontologies*, Technische Universität Berlin, 2007, pp. 27–38.

[18] R. Hennicker and A. Klarl, “Foundations for ensemble modeling–the helena approach,” in *Specification, Algebra, and Software*, Springer, 2014, pp. 359–381.

[19] H. Zhu and M. Zhou, “Role-based collaboration and its kernel mechanisms,” *Systems, Man, and Cybernetics, Part C: Applications and Reviews, IEEE Transactions on*, vol. 36, no. 4, pp. 578–589, 2006.

[20] M. Baldoni, G. Boella, and L. Van Der Torre, “Powerjava: Ontologically founded roles in object oriented programming languages,” in *Proceedings of the 2006 ACM symposium on Applied computing*, ACM, 2006, pp. 1414–1418.

[21] D. Riehle and T. Gross, “Role model based framework design and integration,” in *Proceedings OOPSLA ’98, ACM SIGPLAN Notices*, Oct. 1998, pp. 117–133.